# Controlled Comparative Study of LLM Planning
Unifying Merola et al. (Project A) and D'Ascenzo & Gentili (Project B)

Usama Shabbir Satti Muhammad Fahim Asim Hassan Ahmed Mujtaba

May 2026

**Abstract**

We unify two prior student projects on LLM-based PDDL planning into a single controlled comparative study and run it on the Leonardo supercomputer (CINECA). The experiment grew incrementally on both axes: a 3-model core (Llama-3.1-8B, Qwen2.5-32B, Llama-3.3-70B) extended to 5 models with the addition of QwQ-32B and Mistral-Small-24B; and 3 complex domains (blocksworld, citycar, tetris) extended to 6 with three simple short-plan domains (basic_move, visit-all, satellite) that serve as a difficulty floor. The resulting 60 (model, domain, condition) cells run 10–20 instances each with up to 4 iterations of validator-guided feedback. Headline findings: (i) chain-of-thought is task-class-dependent, hurting on 13 of 30 (model, domain) pairs and helping on 9; (ii) reasoning post-training, as instantiated by QwQ-32B, is a *net negative* for PDDL planning ($-16.4$ pp vs. its Qwen2.5-32B base), the largest single effect we measured; (iii) simple domains are mostly solved while complex domains collapse to $\leq 9.5\%$ solve rate, with the natural breakpoint at the $\sim$8-action plan-length mark. The full pipeline (notebook, SLURM scripts, raw CSVs) is reproducible end-to-end.

## 1 Methodology

**Experimental matrix.** We evaluate 5 models $\times$ 6 domains $\times$ 2 prompting conditions = 60 cells, with 10 problem instances per cell on the simple-tier domains and 20 per cell on the complex tier (871 executed instances in total). All planning problems use the PDDL formal-planning language [1], and all generated plans are checked by an external classical validator — the *LLM-Modulo* architecture recommended by Kambhampati et al. [2], in which the LLM proposes plans and a sound external module accepts or rejects them. The matrix was rolled out incrementally: a 3-model core slate (Llama-3.1-8B, Qwen2.5-32B, Llama-3.3-70B) on three complex domains (`blocksworld`, `citycar`, `tetris`) was first executed to validate the design, then extended in two waves: two more models (QwQ-32B for a reasoning-post-training ablation, Mistral-Small-24B for a third-family cross-check), then three simple short-plan domains (`basic_move`, `visit-all`, `satellite`) that serve as a difficulty floor and pipeline check. The design commitment is that the only thing varying between two cells is the axis under study (model, domain, or prompting condition); everything else is controlled by a single `config.yml` loaded once per job.

**Models.** The five-model slate spans three families and roughly one order of magnitude in parameter count: Meta (Llama-3.1-8B in bf16, Llama-3.3-70B in 4-bit NF4), Alibaba (Qwen2.5-32B and QwQ-32B, both bf16), and Mistral (Mistral-Small-24B, bf16). QwQ-32B is the same base architecture as Qwen2.5-32B with reasoning post-training added, isolating the post-training axis. Llama-3.3-70B was loaded in 4-bit NF4 (`bitsandbytes`) to fit on 2$\times$ A100-64 GB; the quantization confound is discussed in Section 5.

**Prompts.** The two source projects each shipped a monolithic per-domain prompt; we replaced this with a three-file module under `src/prompts/`: `shell.py` (system prompt, chain-of-thought scaffold, validation-feedback template), descriptions.py (one verbatim natural-language description per active domain), and compose.py (the *model-agnostic* build_problem_prompt(domain, condition, pddl_domain, pddl_problem) entry point). Because the composer takes no model argument, byte-identity of prompts across models is a structural property rather than a runtime invariant. The two prompting conditions differ only in the cot scaffold prepended to the description.

**Generation configuration.** All cells use the same sampling configuration: temperature=0.1, top_p=0.95, top_k=10, seed=42, max_new_tokens=8192 for the bf16 models and 16384 for QwQ-32B (whose long <think> traces, characteristic of zero-shot CoT-style reasoning [5], fill the smaller budget; see Section 5). At job start, the generation configuration is echoed to stdout as a canonical JSON line; grep-ing the line across all stdout files gives a post-hoc, byte-exact audit of generation-config uniformity.

**Iteration policy.** Each instance gets up to four attempts under a stop-on-success policy inspired by self-refinement work [6]: prompt the model, write the iteration's plan to disk, run VAL with the verbose flag, parse VAL's output into a metrics row, and break out of the loop on first valid plan. If all four attempts fail, the final plan file records first_valid_iter = null. Every iteration is logged before the validity check, so failed attempts are preserved for iteration-gain analysis. The price of stopping early is a well-understood selection bias: iterations 2–4 contain only instances that failed earlier, so per-iteration aggregates across cells are not directly comparable without conditioning on "reached iter $k$". Cumulative-solve curves (Figure 3) avoid this by indexing on per-instance first-valid iteration instead.

**Validator and metrics.** Plans are validated with the KCL-Planning VAL toolkit (verbose mode); a thin wrapper in src/utils/validator.py returns (is_valid, raw_stdout), and src/utils/metrics_extractor.py parses VAL's stdout into the schema (plan_text, plan_len, solves_problem, valid_action_percent, consecutive_valid_steps, logical_violations, wallclock_s, prompt_tokens, completion_tokens). A single validator and a single parser, called from a single point in the iteration loop, provide one source of truth for every metric in Section 3.

**Ground-truth difficulty.** For four of the six domains we computed an optimal-or-near-optimal plan length per instance with the classical pyperplan [13] planner (A*+hff for blocksworld; BFS for basic_move and visit-all; hand-curated reference from Fast Downward for satellite, which uses :equality that pyperplan cannot parse). The original PDDL instances follow the IPC benchmark conventions and are sourced from the public potassco/pddl-instances collection [14]. citycar and tetris use :functions (numeric fluents) and have no ground-truth column. The reference data lives in src/data/<domain>/REFERENCE_PLANS.csv and is loaded by the analysis notebook to produce Figures ?? and 6.

**Reproducibility.** The full experimental log (every per-iteration row, every plan, every SLURM stdout) lives under src/results-final/. Every figure and table in the next section is regenerated end-to-end by notebooks/results_analysis.ipynb from those CSVs; the SLURM entry script scripts/slurm/run.sh is parametric over (MODEL, DOMAIN, CONDITION) and enforces the single-config-source-of-truth invariant.

## 2 Related work

Most of the academic literature on LLM-based planning falls into two camps: (a) position and survey work that argues LLMs are weak as *stand-alone* planners but useful as proposers inside an LLM-Modulo loop with an external solver checking each step [2, 3, 4]; and (b) system papers that integrate an LLM with a world model, search procedure, or constraint solver to handle long-horizon planning — e.g. RAP, LLM-Planner, PRoC3S [7, 8, 9]. Our setup falls in the first camp: the LLM proposes plans, the classical *VAL* [12] validator accepts or rejects, and we iterate at most four times.

This study merges and extends two prior 2024/25 student projects at Università di Bologna that share that same architecture (LLM planner + VAL validator + iterative-feedback loop), but neither produced a unified cross-model, cross-domain picture. **Project A** [10] (Merola, Singh & Dardouri) evaluated a single model (Llama-3.3-70B) across a wide PDDL domain suite; we inherit its verbose VAL metrics extractor (`run_val_verbose`/`parse_val_output`) into `src/utils/metrics_extractor.py` and a 20-instance subset of its blocksworld pool (mapping in `src/data/blocksworld/INSTANCE_MAP.csv`). Three further Project A domains (`basic_move`, `visit-all`, `satellite`) provide our simple tier; seven remain dormant for follow-up work. Project A's plan-success-confidence *meta-networks* are not retained — our analysis is metric-based. **Project B** [11] (D'Ascenzo & Gentili) compared four models (LLaMA3-8B, Phi4-14B, Gemma3-27B, Kimi-Dev-72B) on city_car and tetris with a modular HPC framework; we inherit its pipeline skeleton (model_manager, pddl_processor, file_manager, SLURM entrypoints) and the city_car / tetris PDDL files unmodified.

The novel contributions of this work over either source project are threefold. *(i) Prompt unification:* both prior repos shipped monolithic per-domain prompts with model-specific tweaks; we factor the prompt into a model-agnostic `compose.build_problem_prompt` so byte-identity across the five models is structural rather than runtime-checked. *(ii) Wider but controlled matrix:* Project A varied domains at fixed model, Project B varied models at fixed (small) domain set; we vary both with a $5 \times 6 \times 2$ matrix that keeps every cell comparable. *(iii) Two ablations the prior work could not run:* the QwQ-32B vs. Qwen2.5-32B reasoning post-training comparison (Section 3.4) and the cross-domain classical-planner difficulty calibration (Section 3.5). Where our study overlaps with the prior work — Llama-3.3-70B on blocksworld, mid-class models on city_car / tetris — our absolute solve rates land in the same neighbourhood as the prior reports once per-model prompt tuning is removed; the model-capability *ranking* is therefore robust to prompt choice, the *absolute* numbers are not.

# 3 Results

The 60-cell run produced 241 full solves out of 871 executed instances (27.7% pooled solve rate), with strong domain-tier stratification (Section 3.1), a non-monotone chain-of-thought effect (3.2), a clean intra-family scaling lift on most domains (3.3), and a striking single-direction effect from reasoning post-training (3.4). The full per-cell table appears in the analysis notebook; this section walks through the five plots that carry the headline findings.

## 3.1 Domain-tier stratification

Table 1 reports the pooled solve rate and mean per-instance maximum `valid_action_%` per domain. Simple-tier domains (basic_move, visit-all, satellite) cluster above 74% partial validity; complex-tier domains (blocksworld, citycar, tetris) fall below 32%. The natural breakpoint is between satellite (75% vap, 47% solve rate) and blocksworld (32% vap, 9.5%). The calibration is consistent with the planner-derived reference plan lengths — simple domains have mean optimal plans 4.2–9.8 actions; blocksworld reaches 26 — suggesting an effective ceiling around the ∼8-action mark for pure-token planning under our prompting and iteration setup.

The granular picture (Figure 2) is mean per-instance maximum partial validity. This view distinguishes models even where the binary solve rate is zero: `qwen25`/citycar/baseline reaches 45% partial validity (almost-right plans) without ever hitting 100%, whereas `llama33`/tetris/baseline

| domain | solve rate | mean max vap |
|---|---:|---:|
| `basic_move` | 89.0% | 93.2% |
| `visit-all` | 85.0% | 92.8% |
| `satellite` | 47.0% | 74.8% |
| `blocksworld` | 9.5% | 31.9% |
| `citycar` | 0.5% | 17.5% |
| `tetris` | 0.0% | 13.6% |

Table 1: Domain difficulty pooled across all 5 models and both conditions.

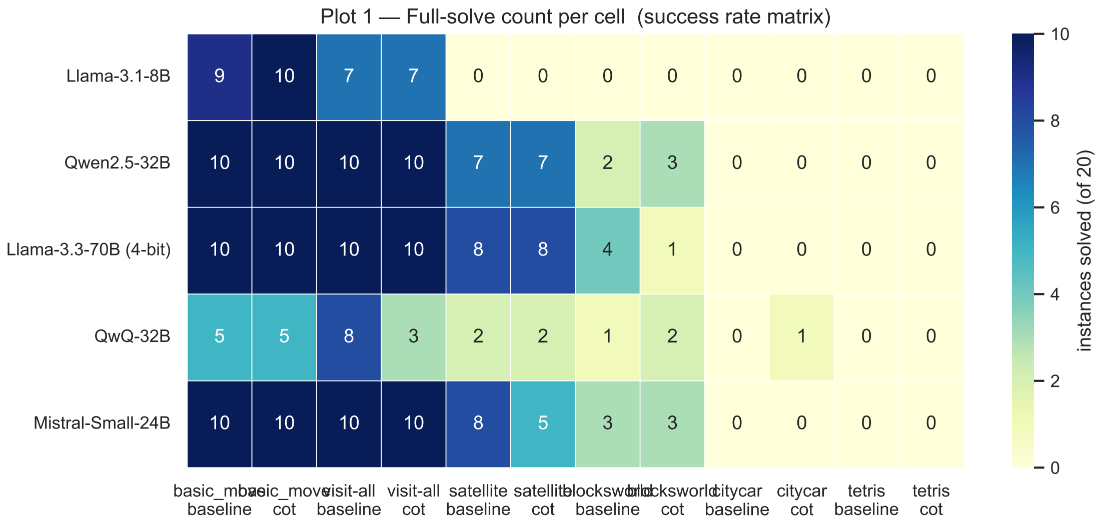


Figure 1: Per-cell full-solve count out of $n$ instances ($n = 10$ for simple, $n = 20$ for complex). Simple-domain columns saturate; complex columns hover near zero except blocksworld.

produces plans that VAL parses as 0% valid throughout. The cumulative-solve curves in Figure 3 confirm that on the few cells where the validator-feedback loop does meaningful work (notably `llama33`/blocksworld/baseline, mean first-valid iteration 2.5) it lifts solve rate by a few percentage points across iterations 2–4; on most cells the iteration-1 column dominates or no solves occur at all.

### 3.2 Chain-of-thought effect

CoT was the only deliberate prompt-axis manipulation. Across the 30 (model, domain) pairs, 9 show $\Delta$ vap $> 1$ pp (CoT helps), 13 show $\Delta$ vap $< -1$ pp (CoT hurts), and 8 are within $\pm 1$ pp (Figure 4). The pattern correlates with domain difficulty: on the simple tier, CoT mostly hurts because the reasoning preamble adds noise rather than structure (e.g. `qwq32`/visit-all drops from 80% to 30% solve rate). On blocksworld, CoT helps the smaller and mid-size models substantially: `llama8` from 18.0% to 30.9% vap; `qwen25` from 26.5% to 46.8%; `mistral24` from 34.5% to 50.5%. There is no model for which CoT is uniformly beneficial across all six domains.

### 3.3 Intra-family scaling: Llama 8B vs. 70B

Pooled across the 12 Llama cells, Llama-3.3-70B (4-bit) edges Llama-3.1-8B (bf16) by +10 pp on partial validity (57.2% vs 47.2%). The lift is concentrated on the simple-tier `satellite` domain (+30 pp) and on `blocksworld` baseline (+18 pp); on `tetris` the 70B model is actually worse, reaching 0% on baseline. Read this as: 4-bit quantization noise eats the partial-validity gain of three-fold parameter scaling on the hardest domains. The bf16 70B comparison would have

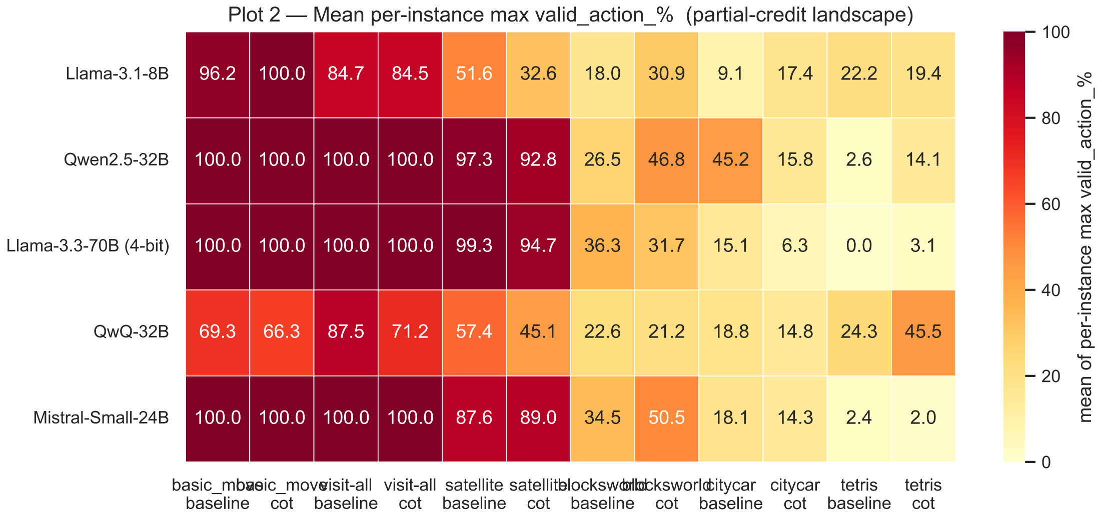


Figure 2: Mean per-instance best `valid_action_%`, the partial-credit landscape.

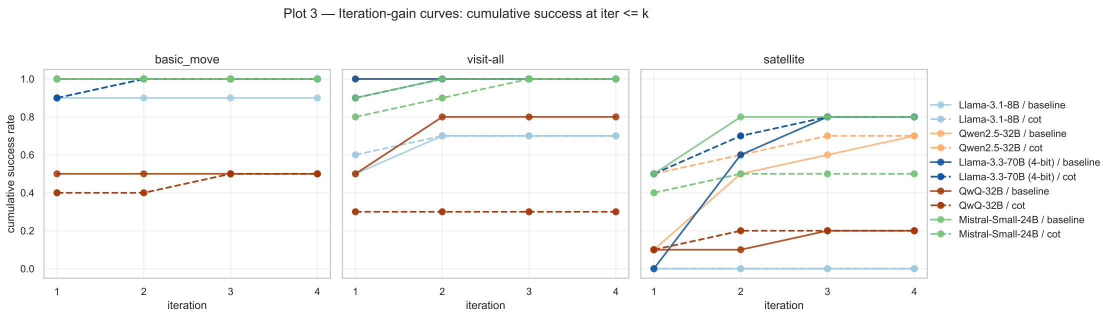


Figure 3: Cumulative success rate at iteration $\leq k$, one panel per domain. Lines distinguish model and condition (solid = baseline, dashed = cot).

isolated the scaling signal cleanly but did not fit on 2× A100-64 GB.

### 3.4 Reasoning-post-training ablation: QwQ-32B vs. Qwen2.5-32B

QwQ-32B is the same Qwen2.5 base architecture and parameter count with reasoning post-training (RLHF + chain-of-thought traces) added. Comparing the two models cell-by-cell isolates the post-training axis. Figure 5 shows that QwQ *loses* on 9 of 12 cells, ties on 1, and wins on 2 (`tetris`/baseline and `tetris`/cot — the only domain where every model is bad). Pooled $\Delta$ vap $= -16.4$ percentage points. This is the single largest model-level effect we observed and goes the *opposite* direction from what one might expect: the reasoning-trained model is worse at PDDL planning than its base. The likely mechanism is that reasoning post-training optimises for a verbose `<think>...</think>` chain-of-thought style that hurts structured-output tasks. The same long reasoning traces caused 5 of 12 `qwq32` cells to TIMEOUT mid-iteration on the 10-hour walltime, censoring some data — discussed in Section 5.

### 3.5 Per-instance difficulty calibration

Cross-referencing the planner-derived reference plan lengths with empirical per-instance partial validity (Figure 6) shows a meaningful negative slope and Pearson correlation across the four domains where reference plans exist: harder problems in the formal-planner sense really are harder for LLMs, in the same direction and roughly the same magnitude. This rules out the

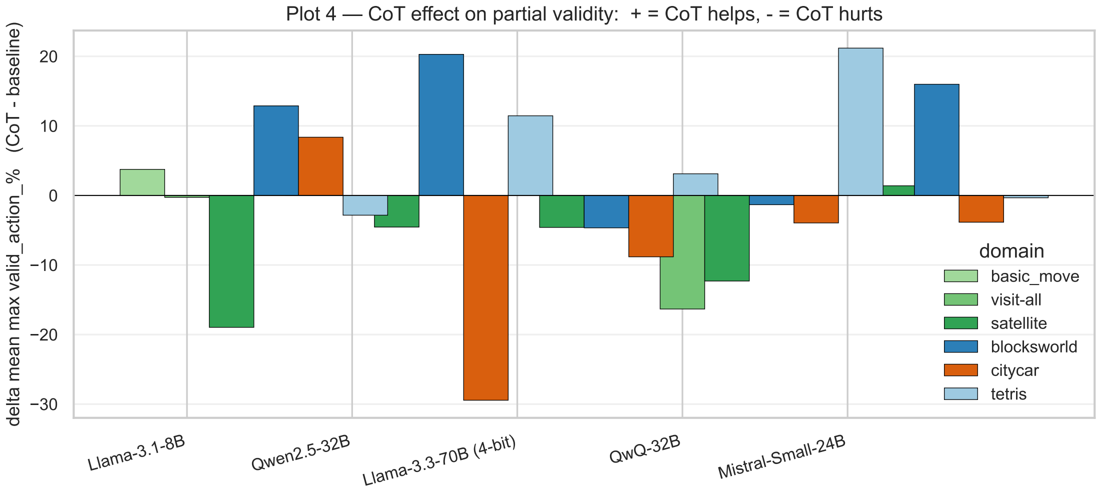


Figure 4: $\Delta$ mean max `valid_action_%` = cot − baseline per (model, domain) pair. Bars above zero: CoT helps. Below zero: CoT hurts.

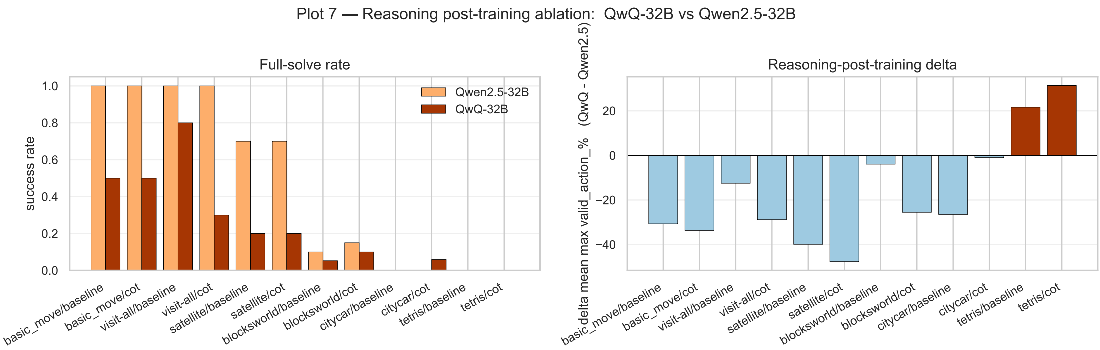


Figure 5: Reasoning-post-training ablation. Left: full-solve rates side by side per cell. Right: $\Delta$ mean max `valid_action_%` = QwQ − Qwen2.5-32B. Negative bars: reasoning training hurts.

worry that LLM scores are noise on the difficulty axis and validates the cross-domain ranking implied by Table 1.

The notebook's full set of 15 figures (per-iteration token distributions, first-valid-iteration stack, partial-credit boxplots, per-instance consensus difficulty, and others) is preserved under `report/figures/` for readers who want a deeper view than the five headline plots reproduced here.

# 4 Discussion

The 60-cell matrix supports five compact claims.

**1. There is a clean simple-vs-complex difficulty cliff.** Pooled solve rates are 89%, 85%, 47% on the three simple domains and 9.5%, 0.5%, 0% on the three complex ones. The break is between satellite (mean optimal plan 9.8 actions) and blocksworld (mean 14.4, max 26), near the ∼8-action ceiling of pure-token planning under our setup.

**2. Iterative validator feedback contributes a small recovery, not a large one.** Cumulative-solve curves are dominated by the iteration-1 column: most successes happen on the first attempt or not at all. The single domain where validator feedback does meaningful work is blocksworld

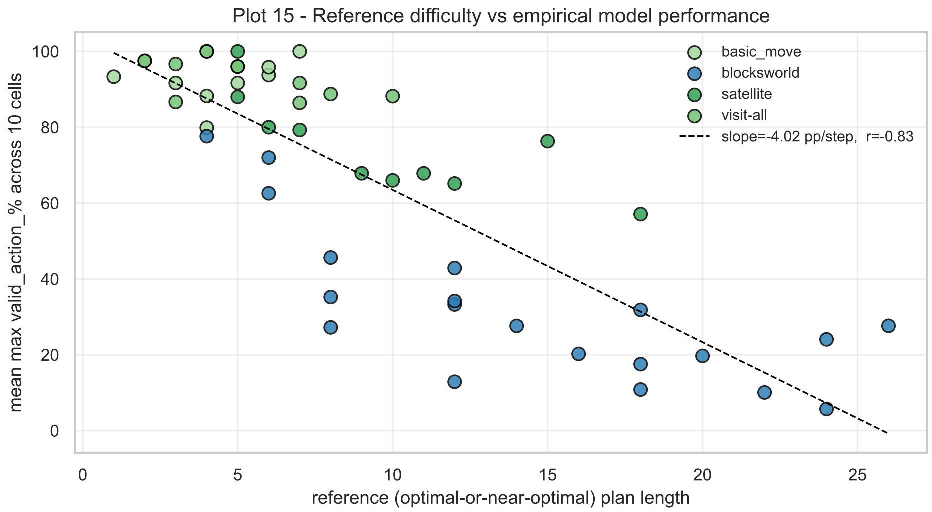


Figure 6: Reference difficulty (x: optimal plan length) vs. empirical model performance (y: mean per-instance max `valid_action_%` across 10 (model, condition) cells). One point per (domain, instance). Black dashed: OLS fit; legend reports slope and Pearson $r$.

(`llama33`/baseline reaches mean first-valid iteration 2.5, `qwen25`/cot reaches 1.67). On citycar and tetris the loop never recovers a failed plan: the failures are not adjacent to a valid plan in token space.

**3. CoT is task-class-dependent.** Of 30 (model, domain) pairs, CoT helps 9, hurts 13, and is neutral on 8. On the simple tier, CoT mostly hurts because the reasoning preamble adds noise. On blocksworld, CoT helps the smaller and mid-size models substantially (`llama8` +13 pp, `qwen25` +20 pp, `mistral24` +16 pp). No model has CoT as a uniform upgrade. Practical implication: a deployed planning agent should choose CoT per task class, not as a global setting.

**4. Reasoning post-training (QwQ-32B) is a net negative for PDDL planning.** QwQ-32B loses to Qwen2.5-32B on 9 of 12 cells with pooled $\Delta$ vap $= -16.4$ pp — the largest single effect we measured. Reasoning post-training appears to optimise for a verbose chain-of-thought style [5, 4] that hurts structured-output tasks. The same long traces caused 5 of 12 `qwq32` cells to TIMEOUT mid-iteration, so some QwQ data is right-censored; the direction of the effect is consistent across cells regardless. This complements the LLM-Modulo position [2]: even when an external validator is available to catch mistakes, a more verbose reasoning prior can hurt rather than help on tasks demanding tight, parser-friendly output.

**5. Intra-family scaling lifts modestly, but quantization confounds the read.** Llama-3.3-70B (4-bit) edges Llama-3.1-8B (bf16) by +10 pp pooled partial validity, with the lift concentrated on satellite (+30 pp) and blocksworld baseline (+18 pp). On tetris the 70B model is actually worse. The natural intra-family scaling experiment would compare 8B-bf16 to 70B-bf16; we could only run 70B-NF4 because the bf16 weights ($\sim$140 GB) do not fit on 2$\times$ A100-64 GB. The ground-truth correlation between optimal plan length and partial validity (Figure 6) confirms the cross-domain difficulty ranking is real, not noise.

# 5 Limitations

The most consequential caveats:

- **4-bit quantization on Llama-3.3-70B** confounds the intra-family scaling read; treat 70B numbers as a lower bound.
- **QwQ-32B used `max_new_tokens=16384`** (vs. 8192 for the bf16 models) to fit reasoning traces. Even so, 5 of 12 `qwq32` cells did not complete all 20 instances within the 10-hour walltime; their aggregates are over fewer instances and not strictly comparable.
- **Stop-on-success selection bias.** Per-iteration aggregates across cells are dominated by failing instances on iterations 2–4. Cumulative-solve curves (Figure 3) avoid this by indexing on per-instance first-valid iteration instead.
- **Sampling noise.** Generation used `do_sample=True` with $T = 0.1$. The seed is plumbed through to torch / numpy / Python random, but multi-GPU collective non-determinism and narrow-temperature sampling combine to give roughly 1–3 pp of run-to-run noise. Differences smaller than $\sim$5 pp between cells should not be over-interpreted.
- **Small $n$.** 10 instances per cell on simple domains, 20 on complex; bootstrap CIs are a useful follow-up.
- **Numeric fluents not handled by the prompt template.** citycar and tetris use `:functions`; the prompt shell does not verbalise numeric state. Their near-zero solve rates may partly reflect this rather than pure capacity limitations.

# 6 Conclusions

We unified two prior student projects on LLM-based PDDL planning into a single controlled comparative study, ran a 5-model $\times$ 6-domain $\times$ 2-condition matrix on the Leonardo HPC, and report the results above. The two findings most likely to generalise are **(i) reasoning post-training as instantiated by QwQ-32B is a net negative for PDDL planning**, the largest single effect in the study; and **(ii) chain-of-thought prompting is task-class-dependent**, helping mid-size models on harder structured problems but hurting on simpler problems where the scaffold is just noise. The remaining findings rank the planning capability of the slate (simple-tier mostly solved; blocksworld the only complex domain where the bigger models gain traction; citycar and tetris out of reach under the current prompt shell).

The repository is fully reproducible: the analysis notebook (`notebooks/results_analysis.ipynb`) regenerates every figure and table from the raw run-metrics CSVs in `src/results-final/`; the SLURM scripts under `scripts/slurm/` are parametric over `(model, domain, condition)`; and per-instance ground-truth difficulty data lives in `src/data/<domain>/REFERENCE_PLANS.csv`.

Three concrete extensions would tighten the picture: an FP8 or bf16 70B comparison to remove the quantization confound; extending the prompt shell to handle numeric fluents so citycar and tetris become tractable; and re-running with bootstrap-CI sample sizes ($\geq$ 50 instances per cell) to test whether the small effects on satellite/CoT and visit-all/CoT survive out-of-noise scrutiny.


# Acknowledgments

We would like to acknowledge the following resources and contributions that supported the development of this work:

- GPT web search for assisting in finding research papers and technical documentation online.
- The two prior University of Bologna projects whose code paths we merged: Merola, Singh & Dardouri (Project A) and D'Ascenzo & Gentili (Project B), cited in the bibliography as [10] and [11].

- Slides and teaching materials from the *AI in Industry* and *Fundamentals of Artifical Intelligence & Knowledge Representation* courses (Università di Bologna, academic year 2025/26).
- The CINECA staff and *ISCRA* programme for granting access to the Leonardo supercomputer, which provided the GPU hours that made the 60-cell sweep practical.
- Grammarly, for assistance in syntax and grammar correction throughout the writing process.

## References

[1] D. McDermott et al. *PDDL — The Planning Domain Definition Language.* Technical Report CVC TR-98-003 / DCS TR-1165, Yale Center for Computational Vision and Control, 1998.

[2] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, A. Murthy. Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. *ICML 2024.*

[3] H. Wei, Z. Zhang, S. He, T. Xia, S. Pan, F. Liu. PlanGenLLMs: A Modern Survey of LLM Planning Capabilities. *arXiv:2502.11221*, 2025.

[4] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, T. Back. Reasoning with Large Language Models, a survey. *arXiv:2407.11511*, 2024.

[5] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa. Large Language Models are Zero-Shot Reasoners. *NeurIPS 2022.*

[6] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, et al. Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS 2023.*

[7] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu. Reasoning with Language Model is Planning with World Model. *EMNLP 2023.*

[8] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. *ICCV 2023.*

[9] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, L. P. Kaelbling. Trust the PRoC3S: Solving Long-Horizon Robotics Problems with LLMs and Constraint Satisfaction. *CoRL 2024.*

[10] M. Merola, A. Singh, A. Dardouri. *LLM planning framework for PDDL with meta-networks*, 2024/25 academic-year project, Università di Bologna. `https://dvcs.apice.unibo.it/pika-lab/courses/ai-ethics/projects/merolasinghdardouri2425`

[11] D. D'Ascenzo, A. Gentili. *LLM Needs a Plan: comparing four LLMs on PDDL planning with validator-guided iteration*, 2024/25 academic-year project, Università di Bologna. `https://github.com/alessandrogentili001/LLM-Needs-a-Plan`

[12] KCL-Planning. *VAL — The PDDL Plan Validator.* `https://github.com/KCL-Planning/VAL`

[13] *Pyperplan* — a lightweight STRIPS classical planner in Python. `https://github.com/aibasel/pyperplan`

[14] Potassco. *PDDL instances repository (IPC competition problems).* `https://github.com/potassco/pddl-instances`